AOA (UT-2)

Date
Page

Q.1 General Method of Dynamic programming List diff. Ex. of it.

→ It's used to solve optimization problems.
→ Breaks down the complex problem into simpler subproblem.
→ find optimal solution to these subproblems.
→ store the results of subproblems. (memoization)
→ reuse them so that same sub-problem is not calculated more than one.
→ finally calculates the result of complex problems.

Ex.

1) Multistage graphs
2) 0/1 Knapsack problems.
3) Optimal binary Search tree.

Greedy Method	Dynamic Programming.
1) Picks the best choice at each step without looking back.	Solves small problems, saves their ans. & combines them for best soln.
2) Quick decision makes.	2) S

Greedy Method.	Dynamic Programming
1) selects the best option in current situation.	– splits the problem into sub then solves it.
2) Makes decision faster as compared to D.P.	– It takes time to solve each problem carefully.
3) Doesn't always give the best answer. it only works for short problems	– Can be use for complex problems and works on
4) Time complexity $O(n \log n)$	– Time complexity $O(n^2)$
5) Uses minimal space since it doesn't save past decisions.	– Need more memory space to store answers
6. Ex. Dijkstra's alg., Kruskal.	– 0/1 Knapsack, longest comm

Longest Subsequence.

		a 1	b 2	b 3	a 4	c 5	d 6	c 7	b 8	a 9
0	0	0	0	0	0	0	0	0	0	0
b 1	0	↑0	↖1	↑1	↑1	←1	←1	←1	↖1	←1
c 2	0	↑0	↑1	↑1	↑1	↖2	←2	↖2	←2	←2
d 3	0	↑0	↑1	↑1	↑1	↑2	↖3	←3	←3	←3
b 4	0	↑0	↖1	↖2	←2	←2	↑3	←3	↖4	←4
b 5	0	↑0	↖1	↖2	←2	←2	←2	←2	↖4	←4
c 6	0	↑0	↑1	↑2	←2	↖3	←3	↖3	↑4	←4
a 7	0	↖1	←1	↑2	↖3	←3	←3	←3	↑4	↖5
a 8	0	↖1	←1	↑2	↖3	←3	←3	←3	↑4	↖5
c 9	0	↑1	←1	↑2	↑3	↑4	←4	↑4	←4	↑5

Greedy Method	Backtracking
1) Makes the locally optimal choice at each step, hoping it will lead to globally optimal soln	– Explores the entire soln space by building a soln step by step an undoing slip.
2) Sequential focused on a single path.	– Uses a depth first search to ensure the soln space tree.
3) Generally faster and more efficient often having time.	– tree is no special set of feasible soln in this method.
4) faster makes only one decision per step no backtracking.	– It consider all possible sequen in order to obtain the optimum soln.

Graph coloring.

Give colors to vertices in a graph so that no two connected points have same color; we assign color to vertices in such a way so that no adjacent vertex have same color.

Algorithm.

```
for j<1 to i<1 do.
if Adjacent (i,j) then.
   if color(i) == color(j++) then.
return 0
  end
 end
end.
return 1.
```

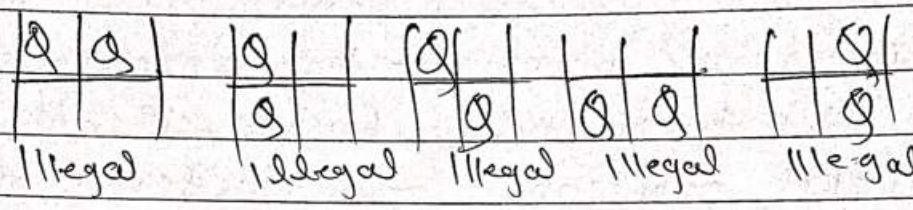
Date
Page

N-Queen Problem.

→ consider a nxn chessboard on which we have to place n queen so that no two queen attack each other by being in same row or colum or diagonal.

If we will solve 2 Queens problem.

It will solved in 2x2 chessboard.



Illegal Illegal Illegal Illegal Illegal.

Algorithm

```
If check Vertex (i)==1 then.
   If i==N then.
   print color (1---n]
else.
 j←1
while (j ≤M) do.
   color (P+1) ← j
   j = j+1
   end
 end
end.
function check Vertex (i).
```

Date
Page

Backtracking	Branch and Bound.
1) Tries one path at a time, like coloring one dot at a time. in graph coloring	- looks at many paths at a time.
2) If two connected dots get the same color, it goes back and picks new color.	- Uses a rule to skip color combinatn. that won't work well
3) Keeps trying until it finds a way to color the whole graph without conflicts.	- stops exploring paths that can't give best soln found so far.
4) ~~Eg~~ Use it when you just need a way to color the graph correctly and graph isn't too big.	- use it when you want the best way to color graph with fewest colors possible.

5)

Q. Explain Naive String matching algorithm

→ It's a simple way to find a small string inside a big string. It checks every possible position in the text to see if the pattern matches or not. ~~an~~ It checks character at a time.

It's like looking for specific word in a book by checking each letter step by step.

Ex. a b c d. — p

a b c d e f g h — T.

a b c d e f g h.

↑ ↑ ↑ ↑

a b c d — all the letters are matched in 1st iteration so we will write.

i++

j++.

Q. 15 puzzled Problem.

It's Invented by Sam Loyd in 1878. In this puzzle there are 15 tiles which are numbered from 1 to 15. The object of this problem is to transformed the arrangement of tiles from Initial arrangement to goal arrangement.

Date
Page

Q. Knuth.– Morris – Pratt algorithm with Ex.

- It avoids, repeated comparison of characters.
- The Basic Idea behind this alg. is to build prefix array sometime this array is also called as π array.
- This prefix array is build using prefix & suffix information of pattern.
- The overlapping of prefix and suffix is used in KMP alg.
- It achieves efficiency of $O(m+n)$ which is optimal & worth steps.

Eg. T = a b a b c a b c a b a b a b d.

a b a b d.

π table.

i	a	b	a	b	d.
	0	0	1	2	0

i

Date
Page

Text : 31 41 59 26 53 5 8 9 7 9 3.

when looking for pattern p=26 Apply pattern q=11

Apply pattern matching

p mod q = 26 mod 11 = 4.

1. 31 mod 11 — 8
2. 14 mod 11 — 3
3. 41. mod 11 — 8

} Not matched.

4. 15 mod 11 — 4
5. 59 mod 11 — 4
6. 92 mod 11 — 4

} Spurious Hits.

7. 26 mod 11 — 4 — matched.

8. 65. mod 11 — 10
9. 53 mod 11 — 9
10. 35 mod 11 — 2
11. 58 mod 11 — 3
12. 89 mod 11 — 1
13. 97 mod 11 — 9
14. 79 mod 11 — 2
15. 93 mod 11 — 5

} Not matched

1) No. of pattern method.
once.

2) Spurious Hits. = 3

3) Shift NO. 6

4) Index no. 7.